

Music Genre Classification: A Machine Learning Approach

Yunus Emre Ulusoy, Semih Doğan, Burak Ala, Alp Eren Gül, Umut Köprülü

I. INTRODUCTION

This report presents work on supervised music genre classification using the GTZAN and FMA Small datasets. The study investigates how far classical feature-based models and lightweight convolutional neural networks (CNNs) can perform when only limited amounts of labeled audio are available. The pipeline starts from the exploratory data analysis of the official FMA and GTZAN features sets, highlighting class balance, strong correlations among descriptors, and substantial overlap between genres in low dimensional projections.

On top of these observations, two families of baselines are implemented: L2-regularized multinomial logistic regression and principal component analysis (PCA) followed by a support vector machine (SVM) with radial basis function (RBF) kernel. These models quantify the discriminative power of handcrafted audio descriptors alone. The analysis then moves to spectrogram based CNNs trained on log-Mel representations computed directly from the raw audio for both datasets, using compact architectures that are compatible with the small-data regime.

Preliminary results indicate that nonlinear models substantially outperform simple linear baselines, and that spectrogram based CNNs can further improve performance, particularly on the more curated GTZAN collection. The report concludes by discussing current limitations related to dataset size, model capacity, data augmentation, and by outlining future directions toward stronger deep learning systems for music genre classification.

II. LITERATURE REVIEW

The classic GTZAN work by Tzanetakis and Cook (2002) [1] is the seminal study that introduced the GTZAN Music Genre dataset and established the modern formulation of the music genre classification problem. This work proposed three families of handcrafted audio features—timbral texture (e.g., MFCC-derived cues), rhythmic content (beat and periodicity), and pitch histograms—computed over 30-second tracks. The authors evaluated statistical pattern-recognition classifiers in two modes: whole-file (offline) and real-time frame-wise. Their reported accuracies were 0.61 for the whole-file case and 0.44 for real-time classification.

In terms of methodology, they first define a timbral texture descriptor computed on short-time frames of roughly 23 ms (512-sample windows) using a Fourier transform, which they derive spectral centroid, spectral rolloff, spectral flux, zero-crossing rate, and Mel-frequency cepstral coefficients, and then they integrate these frame-level measures over a 1

second “texture window” by taking means and variances. Rhythmic content is extracted by passing the audio through a wavelet transform into octave-spaced sub-bands. After the process, they take the amplitude envelopes for pitch content, and they adapt the multipitch detection method of Tolonen and Karjalainen, splitting the signal into low and high bands. Those are combining half-wave rectified and low-pass filtered envelopes, and again using an enhanced autocorrelation to emphasize fundamental periods while suppressing simple multiples, accumulating the salient lags into pitch histograms in a MIDI-pitch space and then folding them into pitch-class histograms. In the whole-file mode, they compute rhythm and pitch features over the entire 30-second excerpt and average timbral texture vectors across all texture windows. ,

They treat genre recognition explicitly as a statistical pattern-recognition problem and train several classic machine learning classifiers on their handcrafted feature vectors. In particular, they use simple multivariate Gaussian classifiers (one Gaussian per genre) and Gaussian Mixture Model (GMM) classifiers whose parameters are learned with the Expectation Maximization algorithm. In addition, they evaluate a non-parametric k-Nearest Neighbors (k-NN) classifier operating directly in the 30-dimensional feature space, using distance to stored training examples rather than an explicit probability model. Together, these methods let them compare generative density models (Gaussian, GMM) and instance-based methods (k-NN) on the same GTZAN feature representation.

Bergstra and colleagues (2006) [2] segment each 30-second clip into short windows, compute MFCC and related low-level descriptors per window, and then aggregate them to the track level by taking statistics such as mean, variance, and sometimes higher moments across time. These vectors are fed into an AdaBoost. MH is a multi-class and hamming loss, which is used in AdaBoost. An MH ensemble that builds many shallow decision trees, each focusing on examples that were misclassified by the previous trees, and gradually forms a strong classifier that defines a complex nonlinear decision boundary in the feature space. The boosting procedure implicitly performs a kind of feature selection, because only features that help reduce error are repeatedly used in the weak learners, and this is one reason the method pushes GTZAN accuracies into the low eighty percent range. Beyond this basic description, Bergstra makes the aggregation and boosting strategy very explicit: each 30-second GTZAN excerpt is first sliced into short, fixed-size frames of 46.44 ms (1024 samples at 22 050 Hz), from which they compute several families of

low-level descriptors, including truncated FFT coefficients, real cepstral coefficients (RCEPS), Mel-frequency cepstral coefficients (MFCC), linear prediction coefficients (LPC), zero-crossing rate, and simple spectral shape measures such as spectral centroid, spread, and rolloff. These frame-level feature vectors are then grouped into non-overlapping segments consisting of m consecutive frames (they systematically explore $m \in \{40, 50, 75, 100, 150, 300, 600\}$, corresponding to segment lengths from roughly 1.8 s up to about 28 s), and for each segment they “aggregate” by fitting independent one-dimensional Gaussians to every feature dimension and taking the sample mean and variance as the segment-level representation, so that each segment is turned into a single, high-dimensional vector of mean/variance statistics rather than a long time series. Each such aggregated segment inherits the genre label of its parent song and becomes one training example for a multiclass AdaBoost.MH classifier, so the effective training set size grows with the number of segments per song, while the classifier still only ever sees fixed-length vectors. In the AdaBoost.MH stage, the input to each weak learner is a d -dimensional segment vector $x \in \mathbb{R}^d$, and the weak learner returns a vector-valued prediction $h^{(t)}(x) \in \{-1, +1\}^K$ over the K genre classes, where each component indicates whether this weak rule “votes for” or “votes against” a given class, and the strong classifier is the weighted sum

$$g(x) = \sum_{t=1}^T \alpha^{(t)} h^{(t)}(x),$$

with the final decision taken as the class index with the largest component of $g(x)$. As weak learners, they use either decision stumps (one-level trees with a single threshold on one feature dimension) or small decision trees; the stump-based variant is denoted AB.Stump and the shallow-tree variant AB. Tree, and both are trained inside AdaBoost.MH by reweighting training examples after each iteration so that misclassified segments receive a higher weight in the next weak learner. Because each stump depends on only a single feature dimension, running AdaBoost.MH with T boosting rounds effectively selects at most T “useful” features out of the full aggregate vector; when $T \ll d$, this acts as an embedded feature-selection mechanism where uninformative or redundant MFCC/FFT/LPC statistics are simply never picked by any weak learner. They also derive the computational complexity of this boosted ensemble: assuming feature values are pre-sorted, training the weak learner on n segments with d features and K classes costs $\mathcal{O}(nKd)$ per round, and the full AdaBoost.MH procedure runs in $\mathcal{O}(nd(KT + \log n))$ time, which scales linearly in the number of segments for fixed T and is therefore more tractable on large music collections than SVMs, whose training cost is quadratic in the number of training points. At test time, each segment of a song is classified independently by the boosted ensemble, and the segment-level discriminant scores $g(x)$ are combined (e.g., averaged or summed) across all segments belonging to a song to produce a final song-level

genre decision, which is the metric reported when they quote GTZAN accuracies. Their systematic segment-length experiments show that segment-level accuracy tends to increase with longer segments, but song-level accuracy first rises and then falls, with an intermediate length around 3.5 s (75 frames) giving the best trade-off between having enough frames to make aggregated means/variances reliable and having enough segments per song to enlarge the training set, and it is precisely in this regime that the AdaBoost.MH ensemble with aggregate MFCC/RCEPS/FFT features reaches GTZAN accuracies in the low 80% range. Compared against alternative classifiers (feed-forward neural networks and SVMs) trained on the same aggregated feature sets and segmentations, the AdaBoost.MH ensembles AB.Stump and AB. Trees are consistently among the top performers, especially when using MFCC or RCEPS aggregates, which reinforces the core methodological point of the paper: for GTZAN-style genre recognition, combining simple frame-level audio descriptors into segment-wise mean/variance aggregates and feeding them into a large-margin boosting ensemble is an effective and computationally efficient way to implement a highly non-linear decision rule in a handcrafted feature space.

A notable CNN study on the GTZAN dataset was done by Zhang (2016) [3]. They use STFT magnitude spectrograms as input and propose two main ideas: (i) combining max pooling and average pooling to get richer statistics from each feature map, and (ii) adding a shortcut (residual) connection to make the network easier to train. The audio is cut into 3-second clips with 50% overlap, and predictions are then combined at the song level. With 10-fold song-level cross-validation on GTZAN, their best model reaches about 87.4% accuracy, which is clearly higher than earlier handcrafted feature methods that usually stay between 60% and 80%. They define two architectures called nnet1 and nnet2. Both take as input a 3-second STFT magnitude spectrogram with around 128 time frames and 513 frequency bins. This 2D time–frequency matrix is treated like an image and passed through several convolutional layers followed by fully connected layers to get the final 10-class genre output. In nnet1, the feature extractor has three convolutional layers. The first layer uses 128 kernels of size 4×513 that move along the time axis, so each kernel sees about 127 ms and all frequencies, and can learn short spectro-temporal patterns. The second and third layers use kernels of size 4×1 , with 128 and 256 feature maps, which increases the temporal receptive field and helps the model learn more abstract patterns such as rhythm fragments or slowly changing timbres. After each convolution, except the last one, they apply a 2×1 max-pooling layer along time. This reduces the time resolution, adds some time-shift invariance, and lowers the computational cost while keeping the strongest activations. A key step in nnet1 is what happens after the third convolutional layer. At this point, they apply global pooling over the time dimension in two ways: global max pooling and global average pooling, with a window equal to the remaining time length (26×1 in nnet1). Then they concatenate the two pooled outputs. In this way, the

final feature vector contains both the maximum activation and the mean activation for each feature map. Their experiments show that this combined pooling works better than using only max pooling or only average pooling. This fixed-length vector is then fed into three fully connected layers with 300, 150, and 10 units. The hidden layers use ReLU activations, and the last layer uses a softmax to produce probabilities for the 10 GTZAN genres. Z. The second architecture, *nnet2*, uses the same general pipeline but adds a residual-style shortcut connection. The output of the first convolutional layer is added directly to the output of the third convolutional layer. This means the third layer learns a residual function $F(x)$, and the final representation is $x + F(x)$, similar to a small ResNet block. This helps optimization and makes training the deeper network easier. In *nnet2*, all three convolutional layers have 256 feature maps, with kernels of size $4 \times (\text{full-frequency or } 1)$. Before the element-wise addition, they use zero-padding in time so that the output of the first layer (125 time steps) and the output of the third layer (119 time steps) have matching time dimensions. After the residual block, they again apply global max pooling and global average pooling over time, concatenate the results, and use the same 300-150-10 fully connected classifier as in *nnet1*. So the main differences between *nnet1* and *nnet2* are the residual shortcut and the slightly different number of feature maps. AA. For training, both networks use Adadelta with the default learning rate of 1.0 and categorical cross-entropy loss. They train with mini-batches of size 50 and use ReLU activations in all convolutional and dense layers. Dropout with a rate of 0.2 is applied to reduce overfitting. In *nnet2*, they also apply batch normalization, which stabilizes the distributions of activations and allows faster and more stable training, especially with a relatively high learning rate. On the data side, each 30-second GTZAN track is split into overlapping 3-second clips with 50% overlap. For each clip, they compute an STFT with 1024-sample windows and 50% overlap, and they use the magnitude spectrum (513 frequency bins) as input to the network. They report that this clip-based setup gives better performance than using a single full-song spectrogram. AB. At test time, the network produces a probability vector for each 3-second clip. For a given song, they sum the probability vectors of all clips from that song and then choose the genre with the highest summed probability as the final song-level prediction. All reported results, including the 87.4% best accuracy, use 10-fold song-level cross-validation, which avoids having clips from the same song in both training and test sets. AC. Their ablation studies clarify which design choices matter. For *nnet1*, using only average pooling gives 84.4% accuracy, which is already better than only max pooling (79.9%). When they concatenate max and average pooling, accuracy increases slightly to 84.8%, which suggests that max and mean activations provide complementary information. For *nnet2*, the residual connection makes the base model stronger: with only max pooling, *nnet2* already reaches 85.0%. When they again combine max and average pooling in *nnet2*, accuracy rises to 87.4%, which is their best result and higher than all previous

GTZAN models they compare with. AD. They also compare their networks to earlier deep learning methods, such as k-max CNN features plus SVM (83.9%), deep neural networks on FFT-based aggregate features (83.0%), and multi-layer invariant representations on log-STFT (82.0%). Their best CNN, which uses STFT magnitude spectrograms, a residual shortcut, and combined global pooling, achieves the highest GTZAN accuracy among these approaches. Overall, the work shows that CNNs on spectrogram “images” can benefit from combining global max and average pooling, and that adding even a single residual block can noticeably improve training and generalization on a small, noisy dataset like GTZAN. For this reason, the Zhang et al. architecture is often used as a strong reference model in later CNN-based music genre classification studies.

In this article, Liu et. al [4] used the GTZAN dataset, which includes 1000 audio clips from 10 musical genres. They introduced a new approach by using a transformer-based model instead of classical CNN layer models. The authors do not use any CNN layers. They mainly rely on self-attention to learn audio features’ global relationships. For instance, CNNs have problems utilizing every layer’s feature data, and RNNs lose the beginning of the data, but with transformers, the model can attend to all time steps and capture patterns that carry music’s characteristics.

The model’s main method is to extract MFCC-based feature sequences from the audio, then feed these sequences into a Transformer encoder. Positional embeddings are added to preserve temporal order, and a special classification token is used so that the Transformer can summarize the entire sequence for genre prediction.

As a result, this Transformer-based model achieves higher accuracy than average CNN models on the GTZAN dataset, showing that global self-attention modeling is effective for music genre classification.

In another article, Shen et. al [5] used the FMA-small dataset, which consists of 8000 audio clips from 8 different music genres. In this project, they presented a new modeling approach based on Functional Data Analysis with continuous mathematical functional features. The authors do not use classical CNN components like Conv2D or MaxPooling with spectrograms. Instead of those, they used basis expansions and functional representations. These functional representations also help them perform better on small, noisy datasets.

As a final result of this model, they reached 68–72% accuracy with the FNN model for the FMA-small dataset. This model outperformed classical Machine Learning models and also showed similar results to simpler CNN models.

In this article, Zhuang et. al [6] used the GTZAN dataset, which includes 1000 samples from 10 different genres. They classified this dataset with their newly developed CNN model, which includes different features like multi-scale convolution, broadcast-based skip connections, dense connections, and feature fusion.

As a short summary of these methods, in multi-scale convolution, they use three different kernels for the same

layer. All these kernels hold different music features such as high and mid frequencies, bass, rhythm, etc. Broadcast-based skip connections help upper layers hold lower layers' features; as a result of this, we preserve lower-layer data. Dense connections provide every layer with the ability to connect to get better gradient flow. Feature fusion combines all layers' features; normally, CNNs only use higher-level layer features.

As a result of the BBNN model, they increased accuracy by almost 4–5%, from 89–91% to 93.9% compared to CNN models, and also performed better than CRNN models by about 2–3%. They utilize mel-spectrogram, Conv2D, ReLU activation, Batch Normalization, MaxPooling, Dropout, Softmax classifier, SGD/Adam optimizer usage, cross-entropy loss, and mini-batch training. These parts are similar to classical CNN models. They also used an 80–10–10 train–test–validation data separation to train the model.

In their article, Ndou, Ajoodha and Jadhav [7] used the GTZAN dataset with 1000 audio excerpts. They are distributed across 10 genres, so there will be 100 songs per genre. The study was done in 3 phases. In Phase A, they extracted the features using MFCC aggregation, spectral features, temporal statistics, chroma features, and beat histograms. In Phase B, they focused on spectral contrast, rolloff, MFCC means, RMS energy, and beat histogram features. In Phase C, they used a modified version of GTZAN, which consisted of 10000 excerpts of 3-second duration. They included features such as chroma, RMS, spectral features, zero-crossing rate, MFCCs, harmony, and tempo. They used traditional machine learning models such as kNN, Linear Logistic Regression, Multilayer Perceptron, Random Forests, Support Vector Machines, and Naive Bayes. They used a CNN with 5 convolutional blocks. Filters were 3×3 . Activation function used is ReLU, and they used 2×2 max pooling with 0.2 dropout rate. CNN is trained on spectrograms and 20 MFCCs.

In Phase A, there were 459 30-second features. Linear Logistic Regression performed the best with 81% accuracy. Random forests had 75.70% accuracy, SVMs had 75.40% accuracy, MLP had 75.20% accuracy, kNN had 72.80% accuracy, and Naive Bayes had 53.20% accuracy. In Phase B, there were 54 30-second features. Here, SVM performed the best with 80.80% accuracy, MLP had 77.30% accuracy, Logistic Regression had 75.80% accuracy, Random Forests had 72.40% accuracy, kNN had 69.70% accuracy, and Naive Bayes had 54.50% accuracy. In Phase C, kNN made a remarkable improvement and performed with 92.69% accuracy. This was due to a couple of reasons. First, since we increased the amount of data by dividing it into 3-second segments, the model was trained better. Also, according to the article, 30-second segments could vary, but 3-second segments can be more precise indicators of the genre. This increase in accuracy is also due to kNN's nature, because abundant data is specifically better for kNN. On the other hand, CNN had an accuracy of 72.40%. This was due to the training data size. Even though they divided the songs into 3-second segments,

this is still a small size for a deep learning algorithm. Another reason is that only 5 convolutional layers are used, which makes the algorithm pretty basic. Also, using 50 epochs for 3-second features may have caused the model to overfit.

The implication of this article is the following: Traditional machine learning approaches can exceed deep learning approaches depending on the dataset and quality of feature engineering. So if the sample size is not too large, implementing well-crafted features instead of creating deep and sophisticated learning algorithms could be a better choice.

In their article, Oramas et al. used a dataset they constructed called “MuMu Dataset” [8]. It consists of 31471 albums and 147295 songs. But what is different in this dataset is that they also have the album cover image, album reviews, and text reviews. What makes this research stand out from others is the fact that it uses audio-based, text-based, and image-based modalities. They also emphasize that combining them together made them achieve better scores in the end.

In the Audio-Based Deep Learning Approach, as input, they used log-compressed CQT spectrograms with 96 frequency bins and 15-second patches. For the CNN architecture, they tested various models with filter sizes 3×3 , 4×96 , and 4×70 . The model has 4 convolutional layers and a max-pooling layer after each. For the output, they implemented 2 configurations. Sigmoid and Cosine. The cosine approach performed consistently better than the Sigmoid because it took the relations between genres into account instead of treating each of them as fully independent genres. In the Text-Based Deep Learning Approach, they took all the reviews of the album and created a single text. They utilized the Vector Space Model in this approach. They also applied a method where they map words to Wikipedia entries and utilize these Wikipedia categories to classify the song. Text performed better than audio with a 0.917 AUC compared to 0.888 AUC because customer reviews are more explicit and signal some cultural meanings that audio cannot convey directly. In the Image-Based Deep Learning Approach, they used ResNet. It was initialized with ImageNet weights. However, this method only had 0.743 AUC.

The significance of this research is the following: Audio is not the most important source for classifying genres because the text-based approach performed way better than audio. So we understand user reviews, tags, mentioning cultural references in the song, etc. can really help classifying the genre. Also, the fact that combining all these 3 models gave us the best performance (0.916 AUC) shows us that different modalities of the song, such as text, audio, and image, complement each other, and they all contribute to something that the others can't.

In his article, Bahuleyan [9] utilized the AudioSet dataset. It consists of 40540 10-second audio clips. They selected 7 genre categories: 8100 Pop music clips, 7990 Rock music clips, 6958 Hip Hop music clips, 6885 Techno clips, 4247 Rhythm and Blues clips, 3363 Vocal music clips, and 2997 Reggae music clips. But the problem with the dataset was that these data are not raw audio clips; they were basi-

cally YouTube videos with start and end time stamps. So, researchers had to develop a method for creating the actual raw data. Some YouTube videos included background noise, sound effects, and non-musical audio elements. So some videos were excluded, possibly creating a sampling bias.

The architecture they used for the deep learning is VGG-16. VGG-16 consists of 13 convolutional layers and 3 fully-connected layers, and 3×3 convolutional filters throughout all layers. A total of 5 convolutional blocks are used, and each block contains a max pooling operation. The number of filters is 64 in the first block, then it is doubled after every block, becoming 512 at the final. So that model can learn more abstract features as it goes through the deeper layers. They used Adam Optimizer, and models were trained with 10 epochs and a batch size of 32. Also, it should be noted that accuracy was maximum at epoch 4, then it started to decrease, indicating the possibility of overfitting. They used 2 methods to prevent the model from overfitting: They used L2 regularization and added a penalty so that really large weights get penalized. They also used dropout regularization with a dropout rate of 0.3. In the traditional approach, they separated features into two subcategories: time domain features and frequency domain features. Time domain features included mean, standard deviation, skewness, kurtosis of amplitude, Zero Crossing Rate, and Tempo (97 features in total). Frequency domain features included MFCCs, 12 Chroma features, Spectral Centroid, Spectral Bandwidth, etc. The learning methods tested are Logistic Regression, Random Forest, XGBoost, and SVM with RBF kernel. As a result, the VGG-16 transfer learning method had 0.63 accuracy, surpassing the best traditional method, XGBoost, with a 0.59 accuracy.

This article's significance is the following: In really large data sizes, like 40000+ here, CNNs with transfer learning outperform traditional machine learning approaches. A significant number lies around 10000 samples for the GTZAN datasets and 40000 for AudioSet.

In their article, Pelchat and Gelowitz [10] used two different music libraries. The first library consisted of 1880 3-minute songs. This dataset was a bit unbalanced because there were 630 Pop songs, but for example, only 27 Electronic songs. So model was biased toward the pop songs and didn't give a good performance on genres with a small amount of data. For data preprocessing, first, they converted the stereo sound to mono sound. Then, they used the SoX command-line to create spectrograms. Each 3-minute song sample is divided into 128 pixel spectrogram segments so that each segment represents 2.56 seconds. So even though the number of songs is very few, the data for training the model was abundant. The 2nd dataset was more balanced. This version consisted of around 200 songs for each genre, which is around 15000 spectrogram pieces per genre. 70% split for training, 20% for validation and 10% for test. In the CNN architecture, they used 4 convolutional layers, but later expanded to 6 layers. Also, they used 2×2 kernels with a stride of 2, and max-pooling is used after every layer.

For initialization, Xavier is used, and for activation, ReLU is used. For regularization, a dropout rate of 0.5 is used. CNN architecture also includes a 4096-unit fully connected layer, and it is trained using the Adam Optimizer.

This study's significance comes from improving the overfitting situation by changing some parameters. They started with a 95% validation accuracy and a 47% test accuracy for 995 songs of 27 genres. Later, they reduced this to 7 genres with 1880 songs and got 62% accuracy. Also, they observed that changing the activation function from ELU to ReLU made the accuracy increase by 5%, accuracy becoming 67%. Also, they switched from Hann to Hamming window for creating spectrograms, and accuracy became 71%. Overall, besides making major changes to parameters and algorithms, a balanced pick of genre distribution and sample size can improve the model's accuracy.

Beyond pure deep learning models, there are also hybrid and optimization-based pipelines for GTZAN. A recent example is the VMD-IWOA-XGBOOST method by Gan (2024) [11], published in MDPI Mathematics. In this work, they first extract features with Librosa, then apply Maximal Information Coefficient (MIC)-based feature selection, and finally use Variational Mode Decomposition (VMD). Both the VMD step and the XGBoost classifier are tuned with an Improved Whale Optimization Algorithm (IWOA). On the GTZAN dataset, this pipeline reaches 85.5% accuracy, 85.4% macro-precision, 86.6% macro-recall, and 85.5% macro-F1. These balanced scores show that hybrid optimization pipelines can be very competitive for music genre classification. In more detail, they design a multi-stage signal processing and optimization pipeline. First, for each GTZAN track, they use Librosa to compute common time-domain and frequency-domain features, such as zero-crossing rate, spectral centroid, spectral roll-off, spectral bandwidth, chroma features, root-mean-square (RMS) energy, delta features, Mel-spectrogram statistics, tempo, and MFCCs. This turns every 30-second audio file into a feature matrix that reflects both short-term timbre and longer-term changes in the signal. On this feature set, they apply the Maximal Information Coefficient (MIC) to measure how strongly each feature (possibly in a nonlinear way) is related to the genre label. MIC is computed by scanning over different 2D grids on the joint distribution of a feature and the labels, then taking the maximum normalized mutual information. The highest-scoring features are kept, and the top five are sent to the next stage.

These top features are then processed by Variational Mode Decomposition. Here, each selected feature is treated as a 1D time series. VMD tries to represent it as a sum of several band-limited components called intrinsic mode functions (IMFs), each capturing a different time scale. This is formulated as a constrained variational problem that uses Hilbert transforms, frequency shifting, and an augmented Lagrangian solved with an alternating direction method of multipliers. The quality of this decomposition depends a lot on hyperparameters, especially the number of modes K and the quadratic penalty factor α . To tune these, the authors use

their Improved Whale Optimization Algorithm, with a fitness function that reflects both the quality of the decomposition and the final classification performance. The IWOA is an extension of the original Whale Optimization Algorithm. It changes the algorithm in three main ways. First, it adds an adaptive inertia weight that changes with the iteration number, which helps the algorithm explore more in the beginning and exploit more near the end. Second, it replaces the fixed logarithmic spiral used in the encircling phase with a variable helix whose shape changes over time, so the search paths become more diverse. Third, it introduces a differential variance factor that slightly perturbs solutions around the current best one, which helps escape local optima, especially in later iterations when the standard WOA would normally get stuck. After VMD decomposes each selected feature with IWOA-tuned K and alpha, the resulting decomposed features are split into training and test sets with an 80/20 split. These are then used as input to an XGBoost classifier, whose own hyperparameters are again optimized by IWOA. For XGBoost, they start from the usual gradient-boosted decision tree framework. The objective function combines a differentiable loss term with a regularization term that controls tree complexity. This objective is approximated with a second-order Taylor expansion in the tree scores, which is the standard XGBoost approach. IWOA is used to search over key hyperparameters such as the number of trees, maximum depth, learning rate, and gamma (the regularization for splits) to maximize validation performance. In this way, the same metaheuristic optimizer jointly tunes both the signal decomposition (VMD) and the classifier (XGBoost), trying to find a globally good match between the features and the model. In their experiments, Gan et al. compare the full VMD-IWOA-XGBOOST pipeline to a wide range of baselines on GTZAN, all using the same Librosa feature set for fairness. The baselines include plain AdaBoost, a back-propagation neural network, an LSTM, gradient boosting decision trees, random forests, standard XGBoost, and a simpler WOA-XGBoost variant. The results show a clear ranking: vanilla XGBoost already beats many traditional models with about 0.665 accuracy and 0.665 macro-F1. WOA-tuned XGBoost improves this to around 0.765 accuracy and 0.767 macro-F1. When they also add VMD and switch from WOA to IWOA, they get the best scores: 0.855 accuracy, 0.854 macro-precision, 0.866 macro-recall, and 0.855 macro-F1. Since macro metrics average across all classes, these results mean the hybrid model helps not only the majority genres but also the minority or harder genres. They also run 10-fold cross-validation and t-tests to show that the improvements of VMD-IWOA-XGBOOST over all baselines are statistically significant. At the same time, they observe that it is actually cheaper to run than the simpler WOA-XGBoost, because the improved optimizer converges faster. Overall, the study shows that combining feature selection, adaptive signal decomposition and metaheuristic hyperparameter tuning around a strong but standard model like XGBoost can rival or even beat many deep models on GTZAN, while keeping the pipeline modular.

III. METHODOLOGY AND DESIGN DECISIONS

A. Exploratory Data Analysis (EDA) - FMA Small

1) *Class Distribution – FMA Small*: Using the cleaned FMA Small feature matrix, we examined the distribution of the eight genres in the FMA Small dataset. The dataset is relatively balanced, with the largest class being *Experimental* at only 13.26%, as seen in the Figure 1 below.

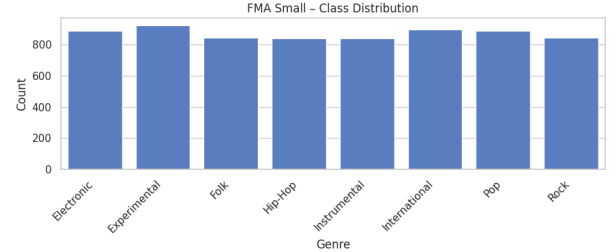


Fig. 1. Class distribution of the FMA Small dataset.

2) *Feature Sanity Checks – FMA Small*: During audio preparation, we encountered failures when attempting to load three FMA Small tracks (099134.mp3, 108925.mp3, and 133297.mp3) using both `pydub` and `librosa`. This was not caused by our processing pipeline: these files are known to be corrupted in the original FMA dataset. According to the official FMA issue tracker (GitHub Issue #41), These tracks contain no audio data at all due to erroneous metadata in the original release. The same issue also notes that six additional tracks in the small subset contain less than 30 seconds of usable audio.

To illustrate how individual features vary across genres, Figure 2 shows one representative example taken directly from the official FMA feature metadata.

Spectral contrast features measure the difference between spectral peaks and valleys across frequency sub-bands; the kurtosis statistic captures how strongly a track alternates between dense and sparse spectral regions. These descriptors often relate to acoustic texture and brightness. We include this feature because it is a typical example of spectral statistics commonly used in music information retrieval and provided directly in the FMA feature set.

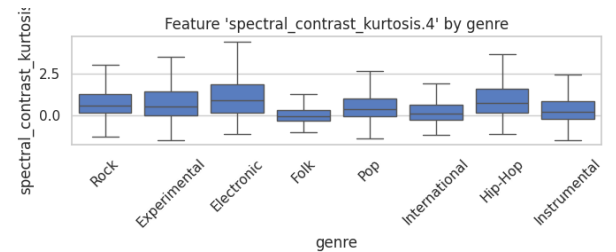


Fig. 2. Distribution of the Spectral Contrast Kurtosis feature across FMA Small genres.

In parallel, we also examined MFCC-based descriptors. MFCCs summarize the overall shape of the frequency content

of the audio over very short time frames. Instead of using raw physical frequencies, the method uses the Mel scale, which compresses higher frequencies and expands lower ones to more closely match how humans perceive pitch. MFCCs are widely used for timbre-related tasks. A timbre-related task is any machine learning or signal-processing task where the objective depends on the type of sound, not the melody or the beat, such as Instrument classification or Genre classification, since genre differences are heavily tied to timbre. (metal = distorted guitars, classical = orchestral timbre, hip-hop = drum-machine timbre, jazz = saxophone/brass timbre). Figure 3 shows the distribution of one such feature, `mfcc_skew.17`. A higher value of this MFCC coefficient generally corresponds to a brighter, more textured timbre with sharper or more percussive high-frequency content. Conversely, lower values indicate a smoother and more harmonically stable spectral envelope, often associated with softer or more sustained instrument sounds.

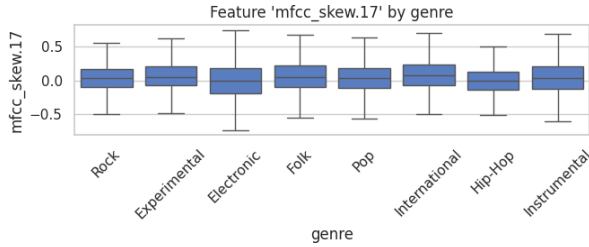


Fig. 3. Distribution of the MFCC skewness (coefficient 17) across genres.

Both examples illustrate an important pattern from our EDA: while individual handcrafted features show genre-dependent tendencies, they also exhibit large inter-genre overlap. This suggests that classical linear models may not fully capture the relationships within the data. These observations support our two-stage modeling strategy: (1) benchmarking classical feature-based models as baselines, and (2) transitioning to spectrogram-based convolutional architectures capable of learning richer nonlinear time–frequency patterns that handcrafted summary statistics cannot fully represent.

3) *Correlation Structure – FMA Small*: The correlation heatmap in Figure 4 reveals strong block patterns among MFCCs, spectral contrast metrics, and chroma features. Many handcrafted features are grouped into strongly correlated families (MFCC, spectral contrast, chroma). This means the feature space is redundant and not independent. Models must therefore be regularized, and these correlations highlight the limitation of purely statistical summaries.

4) *PCA Projection – FMA Small*: The two-component PCA projection in Figure 5 explains only 19.5% of the total variance, indicating that the structure of the handcrafted feature space is highly high-dimensional and not well summarized by a linear projection. Unlike some genres that form partially concentrated regions, the *Experimental* class appears widely dispersed across the PCA plane. This reflects the broad

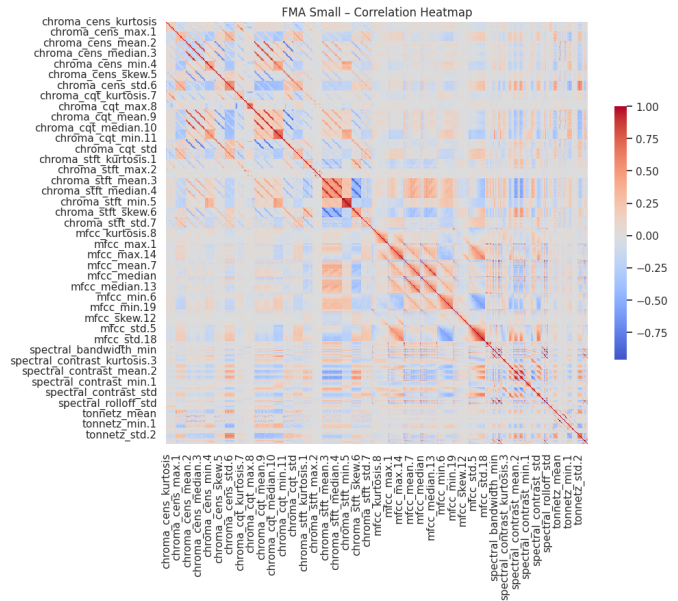


Fig. 4. Correlation heatmap of audio features in FMA Small.

stylistic range contained within the “Experimental” label in FMA Small. As a result, the handcrafted features capture a high degree of internal variability for this genre, causing it to spread across multiple directions of the principal component space. Most genres overlap substantially in the PCA plane. This suggests that the differences between genres are distributed across many dimensions and are likely nonlinear in nature. Consequently, relying solely on handcrafted summary features limits the ability of linear models to capture genre-specific structure.

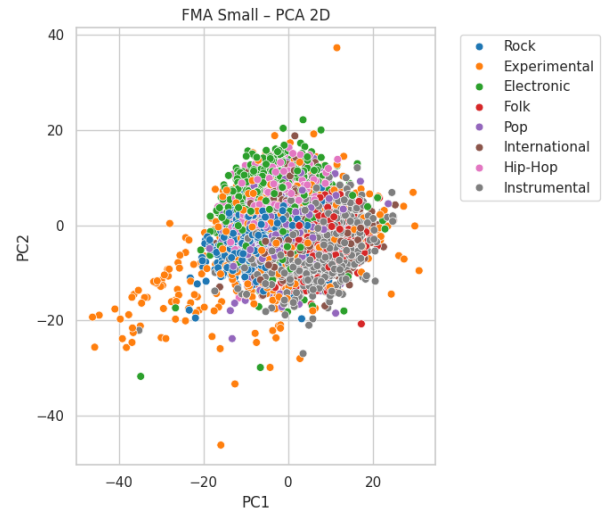


Fig. 5. Two-dimensional PCA projection of FMA Small features.

5) *Data Splitting – FMA Small*: A stratified 80–10–10 split produced:

- 5,556 training samples,

- 695 validation samples,
- 695 test samples,

preserving genre proportions exactly.

B. Exploratory Data Analysis (EDA) – GTZAN

1) *Class Distribution – GTZAN*: GTZAN provides ten genre labels, each originally containing 100 tracks. After loading the pre-extracted feature set and aligning track IDs, the class distribution remains nearly uniform, making GTZAN more balanced than FMA Small. Figure 6 displays the resulting distribution.

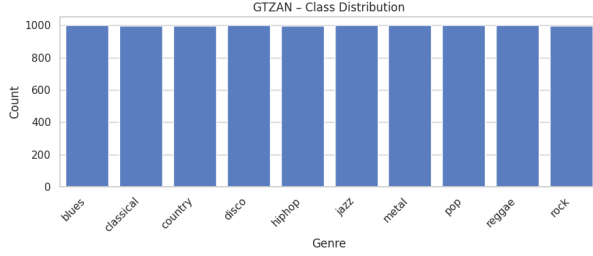


Fig. 6. Class distribution of the GTZAN dataset.

2) *Feature Sanity Checks – GTZAN*: As with FMA Small, the GTZAN feature matrix contains **no missing values**, and all handcrafted audio descriptors fall within expected ranges. GTZAN is known to exhibit lower internal noise and fewer labeling ambiguities compared to FMA, and this is reflected in clearer per-genre patterns across the features.

To illustrate how handcrafted audio descriptors vary across GTZAN genres, Figure 7 shows the distribution of the mean value of the third MFCC coefficient (`mfcc3_mean`). As discussed previously in the FMA Small analysis, lower-order MFCCs capture broad characteristics of the spectral envelope, often relating to overall spectral slope or brightness. In GTZAN, this feature reveals clearer genre-dependent tendencies than in FMA Small, reflecting the dataset's lower internal variability. Nevertheless, considerable overlap between genres persists, indicating that the separability achievable using handcrafted features alone remains limited.

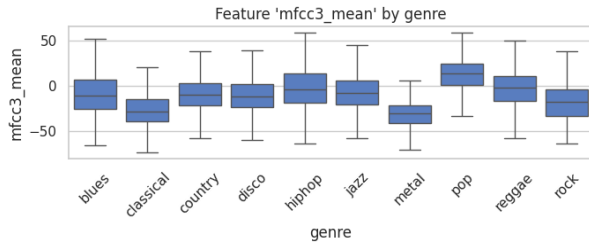


Fig. 7. Distribution of the MFCC3 mean feature across GTZAN genres.

Compared to FMA Small, GTZAN genres tend to exhibit slightly cleaner separation for certain timbral or spectral features.

3) *Correlation Structure – GTZAN*: The correlation heatmap in Figure 8 shows block patterns similar to those observed in FMA Small. MFCC coefficients form a strongly correlated group, spectral contrast features correlate internally, and chroma features cluster together. These structures again confirm that handcrafted features contain substantial redundancy.

The intra-family correlations appear to reflect recording conditions in GTZAN. As with FMA, this redundancy reinforces the need to use regularization when training classical feature-based classifiers.

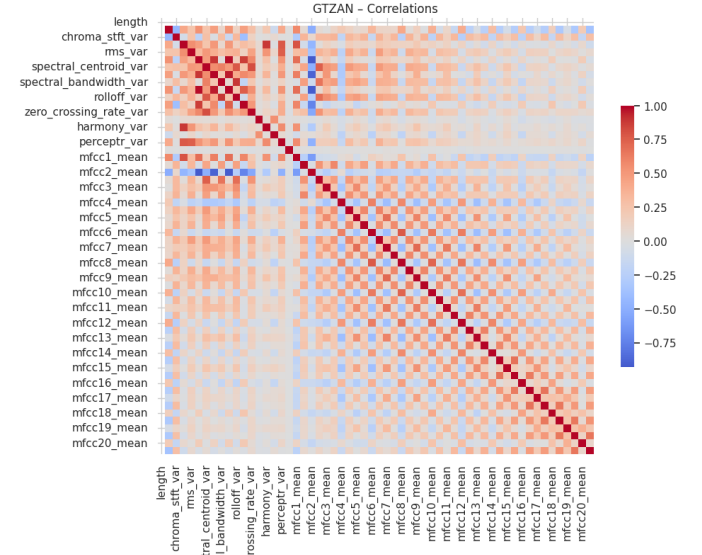


Fig. 8. Correlation heatmap of handcrafted audio features in GTZAN.

4) *PCA Projection – GTZAN*: The two-component PCA projection for GTZAN, shown in Figure 9, explains 33.6% of the total variance—considerably higher than the 19.5% observed for FMA Small. This indicates that GTZAN's handcrafted features contain a more compact and predictable structure, likely due to its cleaner genre definitions and lower internal variability.

In the PCA plane, several genres form visibly tighter regions (e.g., *Rock*, *Metal*, *Jazz*), while genres still exhibit overlap. The clustering behavior suggests that the GTZAN genres possess more consistent timbral patterns, making the dataset more favorable for feature-based classification compared to FMA Small.

Despite this improved structure, substantial inter-genre overlap remains, reinforcing the fact that genre boundaries are distributed across many dimensions. As with FMA, this limitation can show that using spectrogram-based CNNs could provide the ability to learn richer nonlinear relationships.

5) *Data Splitting – GTZAN*: We applied a stratified 80–10–10 split to preserve the uniform class distribution. This yields:

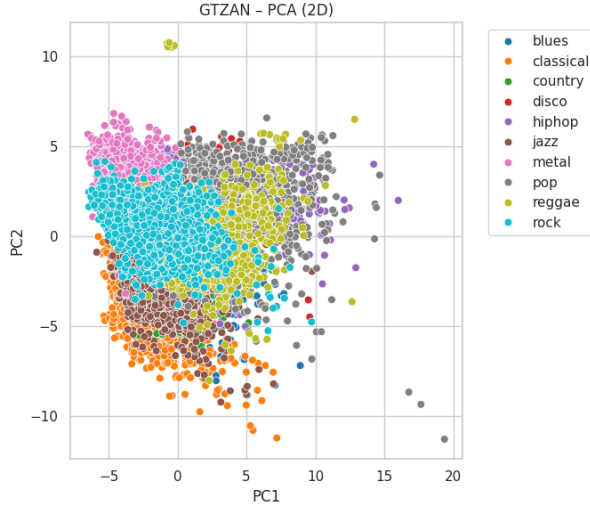


Fig. 9. Two-dimensional PCA projection of GTZAN features.

- 7,992 training samples,
- 999 validation samples,
- 999 test samples.

Because GTZAN is balanced across all genres, stratification preserves its uniformity exactly.

C. Baseline Feature-Based Models

Before introducing convolutional architectures, we establish classical baselines using handcrafted audio descriptors. These models must provide a lower bound on achievable performance without learned time-frequency representations and help quantify how much nonlinear structure exists in the feature space.

1) *Modeling Strategy*: We evaluate two complementary baseline classifiers:

- **L2-Regularized Logistic Regression (LR)**, a linear discriminative model that tests whether the handcrafted features are linearly separable.
- **PCA + SVM with RBF Kernel (SVM-RBF)**, a nonlinear classifier capable of modeling curved decision boundaries in a reduced feature space.

These models serve as interpretable benchmarks against which our future spectrogram-based CNNs can be compared.

2) *Design Decisions: Feature Standardization* All numerical descriptors are standardized using z-score normalization:

$$x' = \frac{x - \mu}{\sigma}.$$

Standardization ensures that MFCCs, chroma vectors, spectral contrast statistics, and rolloff features contribute on comparable scales. Both Logistic Regression and SVMs are sensitive to feature magnitude, making this step essential.

Stratified Data Splitting Both datasets are split into 80–10–10 train/validation/test partitions using stratification to preserve genre proportions. This guarantees that evaluation metrics reflect true generalization rather than sampling artifacts.

Regularized Logistic Regression Logistic regression with an L2 penalty solves:

$$\min_w \|w\|_2^2 + C \sum_{i=1}^n \log(1 + e^{-y_i w^\top x_i}),$$

where C controls the strength of regularization. Regularization discourages overfitting in the high-dimensional handcrafted feature space, especially important given the strong correlations observed in the previous section.

We evaluate $C \in \{0.01, 0.1, 1, 10, 50\}$ using five-fold cross-validation on the training split to select the optimal regularization level.

Principal Component Analysis (PCA) Because handcrafted descriptors form tightly correlated families (MFCCs, spectral contrast coefficients, chroma features), we apply PCA before training SVMs. PCA projects the data into a lower-dimensional subspace:

$$X_{\text{PCA}} = XW_k,$$

where W_k contains the top k eigenvectors of the feature covariance matrix.

Dimensionality reduction reduces redundancy, speeds up training, and stabilizes the RBF kernel.

SVM with RBF Kernel The SVM with a radial basis function kernel computes similarities allowing nonlinear decision boundaries. We perform a two-stage grid search:

- **Stage 1 (coarse)**: broad exploration over PCA components, C , and γ .
- **Stage 2 (fine)**: refined search centered around the best coarse-stage parameters.

All computations for scaling, PCA, LR, and SVM were performed on GPU using RAPIDS cuML.

3) *Experiment on FMA using Logistic Regression*: Table I summarizes the performance extracted from our experiments. The model reaches a test accuracy of **53.7%**, far above the majority-class baseline (13.26%) as proposed in the initial hypothesis.

TABLE I
FMA SMALL — LOGISTIC REGRESSION RESULTS

Class	Precision	Recall	F1
Electronic	0.60	0.64	0.62
Experimental	0.47	0.39	0.43
Folk	0.58	0.58	0.58
Hip-Hop	0.56	0.60	0.57
Instrumental	0.55	0.50	0.53
International	0.62	0.63	0.62
Pop	0.34	0.38	0.36
Rock	0.60	0.58	0.59
Accuracy	0.5367		
Macro Avg	0.54	0.54	0.54
Weighted Avg	0.54	0.54	0.54

These results confirm that handcrafted descriptors contain some genre-discriminative signal, but the linear model struggles with classes that overlap heavily in feature space.

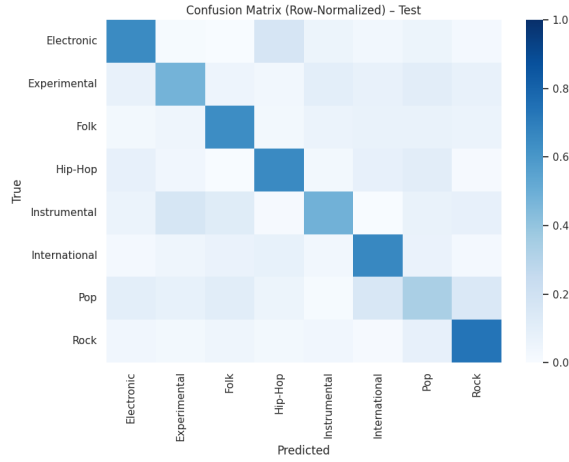


Fig. 10. Row-normalized confusion matrix for the Logistic Regression classifier on FMA Small.

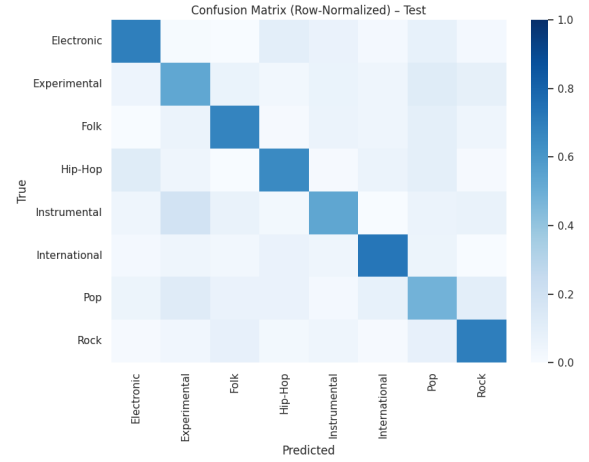


Fig. 11. Row-normalized confusion matrix for the PCA + SVM (RBF) classifier on FMA Small.

4) Experiment on FMA using PCA + SVM (RBF) Baseline:

To allow nonlinear decision boundaries while managing FMA's high dimensionality, we evaluate an SVM with an RBF kernel after PCA compression:

- **Standardization:** Identical to the logistic baseline.
- **Dimensionality reduction:** $k \in \{64, 128, 256\}$.
- **Kernel SVM:** RBF kernel with C and γ searched on logarithmic grids.
- **Model selection:** stratified 5-fold CV on the training set.

This design is motivated by several observations from our EDA: (i) many FMA features are strongly correlated, suggesting redundancy; (ii) PCA captures a moderate amount of variance in the first 100–200 components; (iii) SVMs with RBF kernels could work as strong baselines when classes are not linearly separable.

a) *Results:* The best configuration (typically around 128 principal components, $C \approx 1$ and $\gamma \approx 10^{-3}$) achieved cross-validation accuracy close to **58%**, slightly outperforming logistic regression. This confirms that nonlinear boundaries help, but handcrafted features still impose a ceiling.

5) Experiment on GTZAN using Logistic Regression:

Table II summarizes the performance of the logistic regression model on GTZAN. The classifier reaches a test accuracy of **72.87%**, substantially higher than the majority baseline, confirming that classical handcrafted audio descriptors contain rich genre-discriminative structure.

These results show significantly stronger performance compared to FMA due to GTZAN's more distinct class boundaries and lower dataset noise.

6) Experiment on GTZAN using PCA + SVM (RBF):

To enable nonlinear decision boundaries while reducing the high dimensionality of GTZAN's handcrafted feature space, we evaluate an RBF-kernel SVM following PCA compression:

- **Standardization:** identical preprocessing as in the GTZAN logistic regression baseline.

TABLE II
GTZAN — LOGISTIC REGRESSION RESULTS

Class	Precision	Recall	F1
Blues	0.6869	0.6800	0.6834
Classical	0.9126	0.9400	0.9261
Country	0.6733	0.6869	0.6800
Disco	0.6804	0.6600	0.6701
Hiphop	0.7013	0.5400	0.6102
Jazz	0.8095	0.8500	0.8293
Metal	0.7798	0.8500	0.8134
Pop	0.8438	0.8100	0.8265
Reggae	0.6348	0.7300	0.6791
Rock	0.5567	0.5400	0.5482
Accuracy		0.7287	
Macro Avg	0.7279	0.7287	0.7266
Weighted Avg	0.7280	0.7287	0.7267

- **Dimensionality reduction:** PCA with components $k \in \{32, 58, 64, 128\}$ explored through staged cross-validation.
- **SVM classifier:** RBF kernel with hyperparameters C and γ searched on logarithmic grids.
- **Model selection:** stratified 5-fold CV on the training set.

This design is motivated by GTZAN's structure: although many features are correlated and compressible, the class boundaries are highly nonlinear, making RBF-SVM a strong candidate for capturing complex spectral-temporal interactions.

a) *Results:* The best model (58 principal components, $C \approx 6.31$, $\gamma \approx 0.025$) has validation accuracy of **90.19%** and test accuracy of **91.89%**, outperforming logistic regression.

These results demonstrate that nonlinear decision boundaries dramatically improve classification performance on GTZAN. The RBF kernel effectively separates genres with overlapping harmonic and rhythmic profiles, achieving a high level of performance using only handcrafted descriptors.

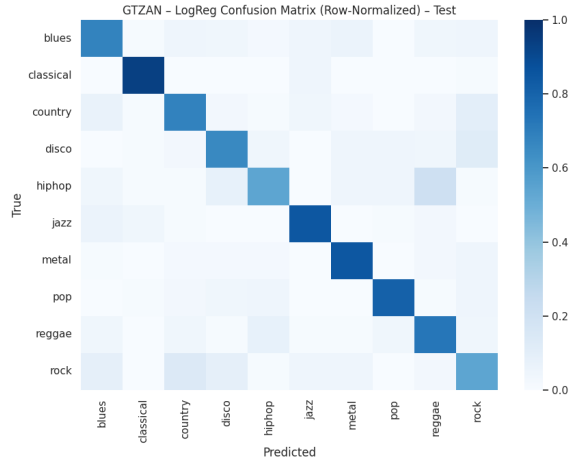


Fig. 12. Row-normalized confusion matrix for the Logistic Regression classifier on GTZAN.

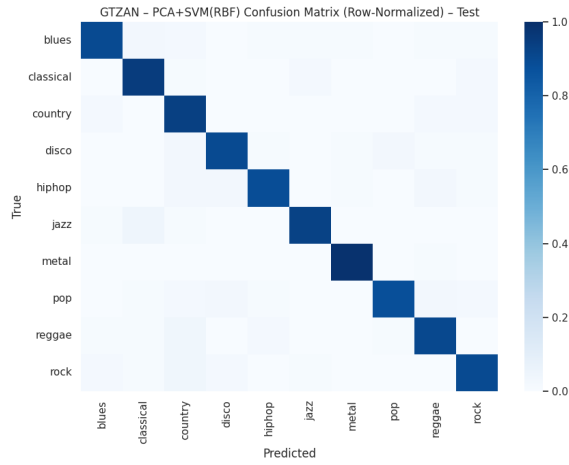


Fig. 13. Row-normalized confusion matrix for the PCA + SVM (RBF) classifier on GTZAN.

TABLE III
GTZAN — PCA + SVM (RBF) TEST RESULTS

Class	Precision	Recall	F1
Blues	0.9375	0.9000	0.9184
Classical	0.8962	0.9500	0.9223
Country	0.8230	0.9394	0.8774
Disco	0.9184	0.9000	0.9091
Hiphop	0.9468	0.8900	0.9175
Jazz	0.9588	0.9300	0.9442
Metal	0.9706	0.9900	0.9802
Pop	0.9565	0.8800	0.9167
Reggae	0.8922	0.9100	0.9010
Rock	0.9091	0.9000	0.9045
Accuracy		0.9189	
Macro Avg	0.9209	0.9189	0.9191
Weighted Avg	0.9210	0.9189	0.9192

D. Deep Learning Models

Deep neural networks for audio usually benefit from very large collections of labeled recordings. In our case, both

GTZAN and FMA Small are relatively limited: GTZAN has only a few hundred clips per class and FMA Small contains 8 balanced genres with fewer than ten thousand tracks. Training high-capacity CNNs on such datasets would almost certainly lead to severe overfitting and unstable optimization. For this reason, the deep models presented in this section are deliberately lightweight and should be viewed as initial trials rather than final architectures. Our goal at this stage is to test whether compact spectrogram-based CNNs can already outperform feature-based baselines under strict data constraints, and to establish a starting point for more advanced architectures and augmentation strategies in future work.

1) *Experiment on FMA Small using CNN*: To evaluate whether deep models can overcome the limitations of hand-crafted audio descriptors, we train a convolutional neural network (CNN) on log-Mel spectrograms computed directly from the raw audio in the `fma_small` dataset. Each audio file is resampled to 22,050 Hz and decoded into a fixed-length 29-second waveform. A 128-bin Mel-spectrogram is computed using a 2048-point FFT and a hop size of 512, followed by conversion to the log-power scale. Each spectrogram is then standardized using per-sample z-normalization. To stabilize training, time dimensions are cropped or padded to a fixed length (1024 frames).

The dataset is split using a stratified 80–10–10 partition, resulting in 5,556 training, 695 validation, and 695 test examples. The network contains four convolutional blocks, each built from two 3×3 convolutions with batch normalization and ReLU activation, followed by 2×2 max-pooling and dropout. The channel widths are:

- **Block 1**: Conv(1→32), Conv(32→32), BN, ReLU, MaxPool(2×2), Dropout(0.10).
- **Block 2**: Conv(32→64), Conv(64→64), BN, ReLU, MaxPool(2×2), Dropout(0.15).
- **Block 3**: Conv(64→128), Conv(128→128), BN, ReLU, MaxPool(2×2), Dropout(0.20).
- **Block 4**: Conv(128→256), Conv(256→256), BN, ReLU, MaxPool(2×2), Dropout(0.25).

After the convolutional stack, an adaptive global average pooling layer (`AdaptiveAvgPool2d(1, 1)`) collapses each of the 256 feature maps to a single value, producing a 256-dimensional embedding. The classifier head consists of a dense layer with 256 units and ReLU activation, followed by dropout (0.30) and a final linear layer projecting to the 8 FMA top-genre classes. The total parameter count printed by the training script is approximately 1.2M.

a) *Results*: The model achieves a test accuracy of **59.1%**, substantially exceeding the classical logistic regression (53.7%) and PCA+SVM ($\approx 58\%$) baselines. Macro-averaged and weighted-averaged scores show similarly consistent improvements.

- **Test accuracy**: 59.1%
- **Macro F1-score**: 0.582
- **Weighted F1-score**: 0.580

These gains indicate that learning time–frequency patterns directly from spectrograms yield more discriminative representations than handcrafted statistics.

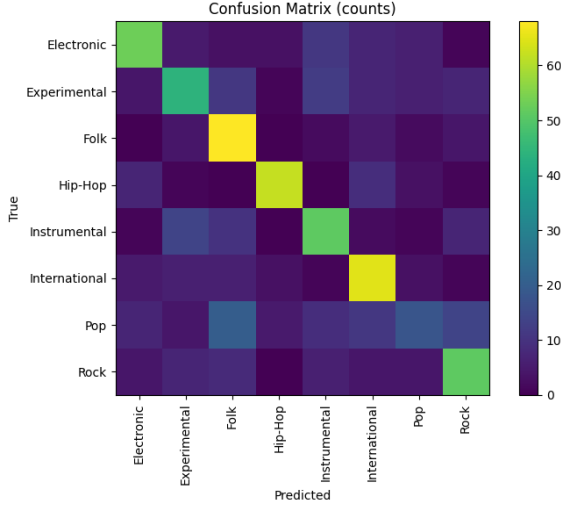


Fig. 14. FMA Small – CNN confusion matrix (counts).

Overall, the CNN establishes a strong deep-learning baseline for FMA Small, surpassing feature-based models and demonstrating the effectiveness of time–frequency representations in a more diverse and less curated dataset than GTZAN.

2) *Experiment on GTZAN using CNN*: To evaluate the effectiveness of spectrogram-based deep learning models, we train a lightweight convolutional neural network (CNN) on 3-second Mel-spectrogram segments generated from the GTZAN `genres_original` audio files. The model uses 64 Mel bands, short-time Fourier transforms with a 1024-point FFT, and per-sample z-normalization Mel log-power spectrograms. Data augmentation (strategy: random gain, additive noise, and random crops) helps the model remain robust to variations in loudness and production quality.

The CNN used for GTZAN follows a lightweight but expressive design similar in style to the FMA model, but scaled to reflect the smaller amount of GTZAN training data. The network consists of four convolutional blocks, each comprising two convolutional layers (3×3 kernels, stride 1, padding 1), followed by batch normalization, ReLU activation, and a 2×2 max-pooling operation. Dropout is applied after every block to improve generalization. Formally, the architecture is:

- **Block 1**: Conv(1→32), Conv(32→32), BN, ReLU, MaxPool(2×2), Dropout(0.1).
- **Block 2**: Conv(32→64), Conv(64→64), BN, ReLU, MaxPool(2×2), Dropout(0.15).
- **Block 3**: Conv(64→128), Conv(128→128), BN, ReLU, MaxPool(2×2), Dropout(0.20).
- **Block 4**: Conv(128→256), Conv(256→256), BN, ReLU, MaxPool(2×2), Dropout(0.25).

Progressive channel expansion (32→64→128→256) increases representational capacity as temporal resolu-

tion decreases. **Dropout scheduling** (0.10→0.25) provides stronger regularization in deeper layers, reducing overfitting on GTZAN's small dataset.

After the convolutional stack, a global average pooling layer condenses each of the 256 feature maps into a single scalar, producing a compact 256-dimensional embedding. This embedding is passed to a two-layer feedforward classifier: a dense layer with 256 units and ReLU activation, followed by dropout (0.3), and a final fully connected layer projecting to the 10 GTZAN classes.

The model contains approximately 0.62 million parameters and is trained for 20 epochs using the AdamW optimizer for efficiency.

a) *Results*: The best model is obtained at epoch 15, reaching a validation accuracy of **77.61%**. On the test set, the model achieves:

- **Test accuracy: 75.00%**
- **Macro F1-score: 0.7466**
- **Weighted F1-score: 0.7545**

Table IV summarizes class-level performance extracted from the evaluation logs.

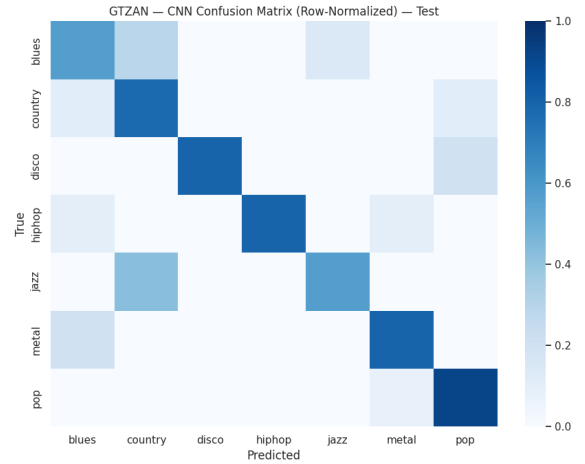


Fig. 15. GTZAN — CNN confusion matrix (row-normalized).

TABLE IV
GTZAN — CNN CLASSIFICATION REPORT (TEST SET)

Class	Precision	Recall	F1
blues	0.7273	0.5714	0.6400
country	0.5000	0.7778	0.6087
disco	1.0000	0.8000	0.8889
hiphop	1.0000	0.8000	0.8889
jazz	0.6667	0.5714	0.6154
metal	0.6667	0.8000	0.7273
pop	0.8000	0.9231	0.8571
Accuracy	0.7500		
Macro Avg	0.7658	0.7491	0.7466
Weighted Avg	0.7806	0.7500	0.7545

The confusion matrices in Figure 15 illustrate the model's strengths and misclassifications. Genres with strong rhythmic

or harmonic signatures (*disco*, *hiphop*, *pop*) show near-perfect recall, while more acoustically diverse categories such as *blues* and *jazz* exhibit higher confusion.

IV. CONCLUSION AND FUTURE WORK

In this progress report, we studied supervised music genre classification on two standard datasets, GTZAN and FMA Small, with the goals of (i) understanding the usefulness and limitations of handcrafted audio descriptors and (ii) evaluating whether spectrogram based lightweight deep models can provide consistent improvements in such small datasets.

TABLE V
SUMMARY OF BEST MODELS ON EACH DATASET.

Dataset	Model	Test Acc.	Macro F1
FMA Small	LogReg	0.537	0.54
	PCA+SVM (RBF)	≈ 0.58	—
	CNN (log-Mel)	0.591	0.582
GTZAN	LogReg	0.729	0.727
	PCA+SVM (RBF)	0.919	0.919
	CNN (log-Mel)	0.750	0.747

The summary in Table V shows a clear contrast between the two datasets. On GTZAN, the PCA+SVM baseline achieves the best performance (test accuracy around 0.92), slightly outperforming the CNN, which suggests that on a small, relatively clean dataset well engineered handcrafted features plus a strong nonlinear classifier are already very effective. On FMA Small, however, the spectrogram-based CNN achieves the highest test accuracy (0.591), edging out both logistic regression and PCA+SVM. This supports our hypothesis that, under noisier and more heterogeneous conditions, even lightweight CNNs trained on log-Mel spectrograms can extract more useful structure than fixed feature vectors, despite the limited data scale.

Overall, the experiments so far support the following conclusions:

- Handcrafted features plus strong nonlinear models (PCA+SVM) are very competitive on clean, relatively homogeneous datasets such as GTZAN.
- On larger and noisier collections such as FMA Small, CNNs on spectrograms provide more flexible representations and yield better performance than linear models on fixed descriptors.
- Confusion matrices reveal systematic genre confusions (for example, *Rock* versus *Pop* in FMA and *Blues* versus *Jazz* in GTZAN), suggesting that some boundaries are ambiguous given the available data.

In the next phase of the project, we plan to:

- **1 - Scale the data:** Utilize larger FMA subsets to study how genre classification accuracy evolves when training CNNs on tens of thousands of tracks instead of only FMA Small and GTZAN.
- **2 - Increase model capacity in a controlled way:** Experiment with heavier architectures such as deeper

CNNs or attention-based models while monitoring overfitting, in order to understand the tradeoff between capacity and generalization at different dataset sizes.

- **3 - Use more advanced data augmentation:** Go beyond simple gain and noise injection, especially for FMA, where label noise and production variability are higher.

A. Success Criteria

The project is deemed successful if it meets the following quantitative targets when finalized, as expressed in the initial report:

- **Accuracy:** $\geq 80\%$ on GTZAN and $\geq 75\%$ on FMA.
- **Macro F1-score:** ≥ 0.80 .
- **Stability:** Cross-validation accuracy std. ≤ 0.03 .
- **Error Analysis:** Clear separation between confusable genres (e.g., rock vs. metal).

REFERENCES

- [1] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.
- [2] J. Bergstra, N. Casagrande, D. Erhan, D. Eck, and B. Kégl, "Aggregate features and AdaBoost for music classification," *Machine Learning*, vol. 65, no. 2-3, pp. 473–484, 2006. [Online]. Available: <https://doi.org/10.1007/s10994-006-9019-7>
- [3] W. Zhang, W. Lei, X. Xu, and X. Xing, "Improved music genre classification with convolutional neural networks," in *Proceedings of the 17th Annual Conference of the International Speech Communication Association (INTERSPEECH 2016)*, San Francisco, CA, USA, September 2016, pp. 3304–3308. [Online]. Available: <http://dx.doi.org/10.21437/Interspeech.2016-1236>
- [4] C. Liu, L. Feng, G. Liu, H. Wang, and S. Liu, "Bottom-up broadcast neural network for music genre classification," *Multimedia Tools and Applications*, vol. 80, no. 5, pp. 7313–7331, 2021.
- [5] J. Shen and G. Xiao, "Music genre classification based on functional data analysis," *IEEE Access*, 2024.
- [6] Y. Zhuang, Y. Chen, and J. Zheng, "Music genre classification with transformer classifier," in *Proceedings of the 2020 4th international conference on digital signal processing*, 2020, pp. 155–159.
- [7] N. Ndou, R. Ajoodha, and A. Jadhav, "Music genre classification: A review of deep-learning and traditional machine-learning approaches," in *2021 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS)*, 2021, pp. 1–6.
- [8] S. Oramas, O. Nieto, F. Barbieri, and X. Serra, "Multi-label music genre classification from audio, text, and images using deep features," 2017. [Online]. Available: <https://arxiv.org/abs/1707.04916>
- [9] H. Bahuleyan, "Music genre classification using machine learning techniques," 2018. [Online]. Available: <https://arxiv.org/abs/1804.01149>
- [10] N. Pelchat and C. M. Gelowitz, "Neural network music genre classification," *Canadian Journal of Electrical and Computer Engineering*, vol. 43, no. 3, pp. 170–173, 2020.
- [11] R. Gan, T. Huang, J. Shao, and F. Wang, "Music genre classification based on VMD-IWOA-XGBOOST," *Mathematics*, vol. 12, no. 10, p. 1549, 2024. [Online]. Available: <https://doi.org/10.3390/math12101549>